Universidad San Carlos de Guatemala

Facultad de ingeniería.

Ingeniería en ciencias y sistemas

Tarea 3: Sistema de Gestión de Notas

PONDERACIÓN: 1 pts

Tiempo estimado: 6 hrs

1. MARCO FORMATIVO

1.1. Valores

Nombre del valor	¿Cómo se aplica en tu laboratorio?
Disciplina	El estudiante demostrará su compromiso y organización para completar todos los requerimientos solicitados de manera clara y ordenada.

1.2. Competencia(s)

Tipo de Competencia	
Competencia General	Comunica Soluciones técnicas complejas a audiencias diversas Utilizando documentación técnica, presentaciones y demostraciones interactivas Adaptando el nivel de detalle según el contexto y audiencia objetivo
Competencia Específica	Analiza Estructuras de datos complejas y algoritmos de ordenamiento Utilizando técnicas de análisis de complejidad temporal y Para optimizar el rendimiento y eficiencia de aplicaciones

1.3. Objetivo SMART

SMART	Definición	Objetivo redactado
Específico (¿Qué?)	El objetivo es concreto y tangible.	Los estudiantes serán capaces de crear operaciones básicas en java, manejo de archivos de texto plano para persistencia y correcto uso de recursividad.
Medible (¿Cuánto?)	El objetivo tiene una medida objetiva de éxito.	Lograr un 100% en resultados al realizar las operaciones solicitadas.

Alcanzable (¿Cómo?)	El objetivo debe ser posible con los recursos disponibles.	Utilizando únicamente las herramientas de laboratorio disponibles en clase.
Realista (¿Para qué?)	El objetivo contribuye a metas más amplias.	Esto ayudara a desarrollar habilidades de programación como de planeamiento y resolución de problemas.
A Tiempo (¿Cuándo?)	El objetivo tiene fecha límite o mejor aún un cronograma de hitos de progreso	Cumplir este aprendizaje en un plazo máximo de 1 día.

2. Material de Apoyo

- Lenguaje de programación Java
- Salida de datos en consola
- Ciclos for, while
- Estructura Try Catch
- Recursividad (método que se llama a sí mismo)
- Algoritmos de ordenamiento (Bubble Sort)
- Manejo de archivos (FileWriter, BufferedReader, FileReader)

3. Actividad

Un profesor necesita un sistema sencillo para registrar las notas de sus estudiantes directamente desde la consola. El sistema debe permitirle agregar estudiantes, ver el listado ordenado por nota, calcular el promedio del grupo y guardar o cargar los datos desde un archivo.

Crea en Java una aplicación de consola llamada **SistemaNotas** que implemente el siguiente menú repetible mediante un ciclo:

```
===== SISTEMA DE NOTAS =====
1. Agregar estudiante
2. Mostrar estudiantes (ordenados por nota)
3. Calcular promedio del grupo
4. Guardar datos en archivo
5. Cargar datos desde archivo
6. Salir
Seleccione una opción:
```

Requisitos obligatorios por opción:

Opción 1 – Agregar estudiante

- Solicitar nombre del estudiante y su nota (0–100).
- Validar con try-catch que la nota sea un número válido dentro del rango. Si no lo es, mostrar un mensaje de error y volver al menú sin agregar nada.
- Almacenar los datos en arreglos paralelos (uno para nombres, otro para notas).

Opción 2 – Mostrar estudiantes ordenados

- Ordenar el arreglo de notas de mayor a menor usando **Bubble Sort implementado manualmente** (no se permite usar Arrays.sort ni Collections.sort).
- Mostrar el listado numerado con nombre y nota de cada estudiante.
- Si no hay estudiantes registrados, mostrar un mensaje indicándolo.

Opción 3 – Calcular promedio

- Calcular la suma de notas usando una **función recursiva** (un método que reciba el arreglo y un índice, y se llame a sí mismo).
- Dividir el resultado entre la cantidad de estudiantes para obtener el promedio.
- Mostrar el promedio con dos decimales.

- Si no hay estudiantes, mostrar un mensaje indicándolo.

Opción 4 – Guardar en archivo

- Guardar todos los estudiantes en un archivo llamado **notas.txt**, uno por línea, en el formato: NombreEstudiante,85.
- Confirmar al usuario que el archivo fue guardado exitosamente.
- Manejar posibles errores de escritura con try-catch.

Opción 5 – Cargar desde archivo

- Leer el archivo **notas.txt** línea por línea y cargar los datos al sistema.
- Si el archivo no existe, mostrar un mensaje indicándolo (no debe lanzar excepción no controlada).
- Confirmar cuántos estudiantes fueron cargados.

Opción 6 – Salir

- Terminar el programa mostrando el mensaje: "Hasta luego.".

Restricciones técnicas

- No se permite usar ArrayList, Collections, ni ninguna clase de java.util para ordenamiento o cálculo. El objetivo es implementar la lógica manualmente.
- El método recursivo de suma **debe estar claramente identificado** con un comentario que explique cómo funciona la recursividad.
- El Bubble Sort **debe estar claramente identificado** con un comentario que explique cada pasada del algoritmo.

Entregable

Un archivo PDF que contenga:

- El código fuente completo con comentarios.

- Capturas de pantalla del programa en ejecución que demuestren cada una de las 6 opciones funcionando correctamente, incluyendo al menos un caso de error controlado en la opción 1.

4. Cronograma

Asignación de tarea	
Entrega de Tarea	

5. Rúbrica de calificación

Redactaras una rúbrica de calificación incluyendo criterios, descripciones y punteos, la siguiente rúbrica solo es a modo de ejemplificación.

CRITERIO	DESCRIPCIÓN	PUNTEO
Agregar estudiante	Registra correctamente nombre y nota; valida con try-catch	15 pts
Mostrar ordenado	Implementa Bubble Sort manualmente y muestra la lista ordenada	20 pts
Promedio recursivo	Calcula la suma mediante una función recursiva correctamente implementada	20 pts
Guardar en archivo	Función Escribe los datos en notas.txt con el formato correcto.	15 pts
Cargar desde archivo	Lee notas.txt y carga los datos sin errores no controlados	15 pts
Comentarios en el	El código tiene comentarios que explican la recursividad y el	5 pts

código	Bubble Sort	
Entregable PDF	PDF con código y capturas de todas las opciones	10 pts
TOTAL		**100 pts**

Reflexión y aplicación práctica

¿Qué aprendí con esta tarea?

¿Cómo crees que aplicarás este conocimiento en otro curso o en la vida profesional?